

Operaciones de Conjuntos en Bridge GeCode

Guía Completa de Sets y Operaciones

Proyecto GNUMBison

2026

Contents

Operaciones de Conjuntos	2
Introducción	2
Operador IN	2
UNION	3
INTERSECT	4
DIFFERENCE	5
SUBSET	6
CARDINALITY	7
Ejemplos Completos	8
Casos de Uso Comunes	10
Combinando con Otros Operadores	11
Notas Importantes	11
Resumen de Operadores	11

Operaciones de Conjuntos

Introducción

Bridge GeCode soporta operaciones avanzadas de conjuntos (sets) que permiten modelar problemas complejos de asignación, permisos, roles, y más.

Sintaxis Moderna

Importante: Los sets literales en expresiones se escriben **sin comillas**:

Correcto: {dev,qa,pm}

Incorrecto: {"dev","qa","pm"}

Nota: En JSON, los valores de variables SÍ llevan comillas por estándar JSON:

```
"value": ["dev", "qa"]
```

Operador IN

Verifica si un elemento pertenece a un conjunto.

Sintaxis

elemento IN {conjunto}

variable IN otra_variable

Ejemplos

Verificar pertenencia de número:

codigo IN {10,20,30,40,50}

Verificar pertenencia de identificador:

rol IN {admin,supervisor,gerente}

Con variable:

usuario IN equipo_autorizado

UNION

Combina dos conjuntos, retornando todos los elementos únicos.

Sintaxis

```
conjuntoA UNION conjuntoB  
variable UNION {elementos}
```

Ejemplos

Agregar roles:

```
equipo UNION {pm,dba}
```

Si equipo = {dev,qa}, resultado: {dev,qa,pm,dba}

Combinar permisos:

```
permisos_base UNION permisos_extra
```

INTERSECT

Retorna solo los elementos comunes entre dos conjuntos.

Sintaxis

```
conjuntoA INTERSECT conjuntoB
```

Ejemplos

Encontrar roles comunes:

```
equipo INTERSECT {dev,qa,admin}
```

Si equipo = {dev,qa,pm}, resultado: {dev,qa}

Servicios activos y permitidos:

```
servicios_activos INTERSECT servicios_permitidos
```

DIFFERENCE

Retorna elementos del primer conjunto que NO están en el segundo.

Sintaxis

`conjuntoA DIFFERENCE conjuntoB`

Ejemplos

Remover elementos:

`equipo DIFFERENCE {qa}`

Si `equipo = {dev,qa,pm}`, resultado: `{dev,pm}`

Tareas pendientes:

`tareas_totales DIFFERENCE tareas_completadas`

SUBSET

Verifica si el primer conjunto es subconjunto del segundo (todos sus elementos están contenidos).

Sintaxis

```
conjuntoA SUBSET conjuntoB
```

Ejemplos

Verificar permisos mínimos:

```
{read,write} SUBSET permisos
```

Retorna true si permisos contiene al menos read y write.

Verificar roles requeridos:

```
roles_minimos SUBSET roles_asignados
```

Comparación:

```
{dev} SUBSET equipo          → true si equipo contiene dev  
{dev,admin} SUBSET equipo    → true solo si equipo contiene AMBOS
```

CARDINALITY

Cuenta la cantidad de elementos en un conjunto.

Sintaxis

```
CARDINALITY(conjunto)
CARDINALITY({elementos})
```

Ejemplos

Contar elementos de variable:

```
CARDINALITY(equipo)
```

Si equipo = {dev,qa,pm}, resultado: 3

Contar literal:

```
CARDINALITY({alice,bob,charlie})
```

Resultado: 3

En restricciones:

```
CARDINALITY(equipo) >= 3
```

```
CARDINALITY(permisos UNION {admin}) <= 5
```

Ejemplos Completos

Ejemplo 1: Control de Acceso

Archivo: ejemplo_acceso.json

```
{
  "precision": 0,
  "variables": [
    {
      "nombre": "permisos",
      "tipo": "set",
      "domain": ["read","write","execute","delete"],
      "value": ["read", "write"]
    },
    {
      "nombre": "autenticado",
      "tipo": "logic",
      "domain": [true, false],
      "value": true
    }
  ],
  "expresiones": [
    "autenticado = true",
    "{read} SUBSET permisos",
    "{write,delete} SUBSET permisos",
    "CARDINALITY(permisos) >= 2"
  ]
}
```

Ejecutar:

```
./bridge ejemplo_acceso.json resultado.json
```

Resultado:

```
{
  "expresiones": [
    {"expression": "autenticado = true", "resultado": true},
    {"expression": "{read} SUBSET permisos", "resultado": true},
    {"expression": "{write,delete} SUBSET permisos", "resultado": false},
    {"expression": "CARDINALITY(permisos) >= 2", "resultado": true}
  ]
}
```


Ejemplo 2: Gestión de Proyectos

Archivo: ejemplo_proyecto.json

```
{
  "precision": 0,
  "variables": [
    {
      "nombre": "equipoActual",
      "tipo": "set",
      "domain": ["dev", "qa", "pm", "ops"],
      "value": ["dev", "qa"]
    },
    {
      "nombre": "equipoNecesario",
      "tipo": "set",
      "domain": ["dev", "qa", "pm", "ops"],
      "value": ["qa", "pm", "ops"]
    }
  ],
  "expresiones": [
    "CARDINALITY(equipoActual) >= 2",
    "CARDINALITY(equipoActual UNION equipoNecesario) <= 4",
    "equipoActual INTERSECT equipoNecesario"
  ]
}
```

Resultado:

```
{
  "expresiones": [
    {"expresion": "CARDINALITY(equipoActual) >= 2", "resultado": true},
    {"expresion": "CARDINALITY(equipoActual UNION equipoNecesario) <= 4", "resultado": true},
    {"expresion": "equipoActual INTERSECT equipoNecesario", "resultado": ["qa"]}
  ]
}
```

Casos de Uso Comunes

1. Control de Acceso y Permisos

```
{admin} SUBSET roles  
permisos UNION {read,write}  
permisos DIFFERENCE {delete}
```

2. Gestión de Equipos

```
CARDINALITY(equipo) >= 3  
equipo INTERSECT roles_senior  
{dev,qa} SUBSET equipo
```

3. Configuración de Servicios

```
servicio IN {http,https,ssh}  
servicios_activos UNION {ftp}  
servicios DIFFERENCE servicios_deprecated
```

4. Recursos Humanos

```
departamento IN {ventas,it,rrhh}  
CARDINALITY(idiomas) >= 2  
certificaciones SUBSET {iso9001,iso27001}
```

5. Inventarios y Productos

```
producto IN stock_disponible  
categorias UNION {electronicos}  
CARDINALITY(productos) <= 100
```

Combinando con Otros Operadores

Con Operadores Lógicos

```
activo = true AND {admin} SUBSET roles  
CARDINALITY(equipo) >= 3 OR urgente = true
```

Con Operadores Aritméticos

```
CARDINALITY(equipo) * costo_persona <= presupuesto  
empleados >= CARDINALITY(departamentos) * 5
```

Con Comparaciones

```
CARDINALITY(servicios UNION {nuevo}) <= 10  
CARDINALITY(permisos) <> 0
```

Notas Importantes

1. **Sets vacíos:** {} es un conjunto válido con cardinalidad 0
2. **Elementos únicos:** Los sets no permiten duplicados
3. **Tipos:** Los elementos de un set son siempre strings o identificadores
4. **Comparación:** Dos sets son iguales si contienen exactamente los mismos elementos
5. **Performance:** Las operaciones de sets son eficientes incluso con conjuntos grandes

Resumen de Operadores

Operador	Símbolo	Tipo Retorno	Ejemplo
IN	IN	logic	x IN {a,b,c}
UNION	UNION	set	A UNION B
INTERSECT	INTERSECT	set	A INTERSECT B
DIFFERENCE	DIFFERENCE	set	A DIFFERENCE B
SUBSET	SUBSET	logic	A SUBSET B
CARDINALITY	CARDINALITY	integer	CARDINALITY(A)